SIMATS ENGINEERING

ASSESSMENT TOOL 2 – DATA INTERPRETATION (MEDIUM)

Course Name: Internet Programming | Course Code: CSA43

Course Outcome CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery (L2).

Student Name: Sreeshanth Basineni
Register Number: 192372238

1. HTTP Response Analysis

Given response:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1.1 Identify any incorrect or invalid fields in the header.

The Content-Length field is invalid because it is set to -150. Content-Length represents the size of the message body in octets (bytes), so its value must be a non-negative decimal integer. The Date field also has a possible weekday/date mismatch: 1 April 2026 is a Wednesday, not Monday. The other fields are syntactically plausible: Content-Type identifies HTML content, and Connection: keep-alive requests a persistent connection in HTTP/1.1.

1.2 Analyze the issue with Content-Length and explain its impact.

A negative content length cannot describe the size of an HTTP response body. A client may reject the malformed response, report a protocol error, or fail to determine the correct message boundary. Incorrect message framing can cause the response to be truncated or interpreted incorrectly and, in some circumstances, can contribute to request/response desynchronization problems. The server should send the actual body size as a non-negative value, or use the framing mechanism appropriate to the protocol.

1.3 Determine whether the status code matches the response correctness.

The status code 200 OK means that the request was successfully handled and the response is being returned successfully. However, it does not make the malformed Content-Length valid. Because the header contains an invalid field, the response is not fully correct even though the status code itself is a valid success code. A server should correct the header before sending the response.

1.4 Suggest corrections for the identified errors.

Set Content-Length to the actual number of bytes in the response body, for example Content-Length: 150 only if the body is exactly 150 bytes. Correct the Date header to Date: Wed, 01 Apr 2026 10:00:00 GMT if that timestamp is intended. Keep Content-Type: text/html when the body is HTML.

Keep Connection: keep-alive if a persistent HTTP/1.1 connection is desired. Example corrected header:
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
The example length must be replaced with the true body size.

2. DOM Structure Analysis

Given structure:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
<body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

2.1 Identify missing or improperly closed elements.

The paragraph element is opened but has no explicit closing tag before the tag. The div, body, head, and html elements are closed. The main structural error is the unclosed paragraph.

2.2 Analyze how the browser will interpret this structure.

HTML browsers use error-recovery rules when markup is malformed. In this case, the parser will generally infer the end of the paragraph when it encounters the closing , producing a DOM that is more complete than the source text. Browser recovery is useful, but it should not be relied on as a replacement for valid markup.

2.3 Determine the impact on rendering and layout.

The page may still display “HelloWorld” because browsers repair many common HTML errors. However, relying on automatic repair can make the DOM harder to inspect and maintain, and can create inconsistent results when more complicated nesting errors occur. Correctly closed elements improve predictable layout, accessibility, and maintainability.

2.4 Suggest corrections to make the DOM valid.

Close the paragraph before closing the div. A corrected version is:

```
<html>
 <head>
  <title>TestPage</title>
 </head>
```

```
  <body>
    <div>
      <p>HelloWorld</p>
    </div>
  </body>
</html>
```

3. GET vs POST in Login System

3.1 Interpret the differences in data transmission between GET and POST.

GET sends parameters as part of the URL, commonly in a query string such as /login?user=admin&pass=1234. These values can appear in browser history, server logs, copied links, and referrer information. POST sends submitted form data in the HTTP request body rather than placing it in the URL. The body is not automatically encrypted; encryption depends on using HTTPS. GET is generally intended for retrieving resources, while POST is commonly used to submit data that changes state or needs to be processed.

3.2 Analyze which method is more suitable for login and why.

POST is more suitable for submitting login credentials because it avoids placing the username and password directly in the URL. This reduces accidental exposure through browser history and URL sharing. POST alone does not make login secure, so it must be combined with HTTPS and secure server-side authentication.

3.3 Identify security risks in using GET for authentication.

Credentials in a GET URL can be saved in browser history, recorded in web server or proxy logs, copied into screenshots or messages, and potentially exposed through referrer data. URLs may also be visible to people looking over a user's shoulder. If HTTP rather than HTTPS is used, data can be intercepted in transit regardless of whether GET or POST is used.

3.4 Suggest best practices for secure login implementation.

Use POST over HTTPS for credential submission; never store passwords in plain text; hash passwords with a modern password-hashing algorithm such as Argon2id, bcrypt, or scrypt with appropriate parameters and salts; validate credentials on the server; apply rate limiting and protections against brute-force attempts; use secure, HttpOnly, SameSite cookies for session identifiers; rotate session IDs after successful login; implement multi-factor authentication where appropriate; avoid logging passwords; and return generic login errors that do not reveal whether a username exists.

4. HTML Code Inspection

Given HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
```

```
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

4.1 Identify missing closing tags.

The <h1> heading is missing </h1>, and the paragraph is missing . The head, body, and html elements have closing tags. The img element is a void element in HTML, so it does not require a closing </img> tag.

4.2 Analyze issues with the <img> tag.

The image tag has a src attribute, but it is missing an alt attribute. Alternative text is important for screen-reader users and is displayed when an image cannot load. The file image.jpg must also exist at the path relative to the HTML document, or the browser will show a broken-image indicator. Width and height attributes can optionally help reserve layout space.

4.3 Determine how browsers will handle these errors.

Browsers usually attempt to repair missing end tags and continue rendering, so the page may appear mostly correct. The parser can infer where a heading or paragraph ends when it encounters later elements. However, the result should not be left to error recovery: malformed markup can make the DOM less predictable, and a missing or incorrect image path means the image will not load.

4.4 Suggest a corrected version of the code.

The following version closes the heading and paragraph and supplies meaningful alternative text. Replace the alt text with a description that accurately represents the actual image.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
  <img src="image.jpg" alt="Description of the image">
</body>
</html>
```

5. Browser–Server Communication Flow

5.1 Interpret each step in the communication flow.

1. User enters a URL: The browser reads the address and identifies the scheme (HTTP or HTTPS), host, and requested path.
2. DNS lookup: The browser or operating system checks available caches and, if needed, asks a DNS resolver to translate the domain name into an IP address.
3. HTTP request is sent: The browser establishes the required connection (including TLS for HTTPS) and sends an HTTP request with a method, path, headers, and possibly a body.
4. Server processes request: The server receives the request, applies routing and application logic, may query a database or other service, and prepares a response.
5. Response is returned: The server sends a status code, response headers, and the response body, such as HTML, CSS, JavaScript, or data.
6. Browser renders page: The browser parses HTML into the DOM, processes CSS, runs JavaScript as needed, fetches linked resources, and paints the page on screen.

5.2 Identify where delays can occur.

Delays can occur during DNS resolution, connection establishment, TLS negotiation, network transmission, server-side processing, database queries, and the download or processing of large HTML, CSS, JavaScript, image, and font files. Rendering can also be delayed by blocking scripts, large stylesheets, heavy JavaScript, or an overloaded device.

5.3 Analyze the role of DNS in the process.

DNS (Domain Name System) translates a human-readable domain name, such as example.com, into an IP address that the browser can use to contact the server. DNS caching reduces repeated lookups and usually improves speed. If DNS is slow, misconfigured, or unavailable, the browser may take longer to connect or may fail to reach the server even when the website itself is running.

5.4 Suggest optimizations to improve performance.

Use caching for DNS and static resources; use a reliable DNS provider; enable HTTPS with efficient connection reuse; use HTTP/2 or HTTP/3 where supported; compress text responses with Brotli or gzip; optimize and appropriately size images; minify CSS and JavaScript; remove unused code; defer non-critical JavaScript; use a content delivery network for geographically distributed users; reduce server and database processing time; enable suitable cache-control headers; and monitor response times. These improvements target different stages of the flow, so performance should be measured to identify the main bottleneck.

Conclusion

The analysis shows that reliable web communication depends on valid HTTP headers, well-formed HTML and DOM structure, appropriate HTTP methods, and efficient browser–server processing. Browsers can recover from some markup errors, but valid code and correct protocol fields make behavior more predictable. For login systems, POST must be paired with HTTPS and sound authentication practices; for performance, optimizations should address the actual source of delay.

Code of Conduct

I, SreeshanthBasineni (Register Number:192372238), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sreeshanth Basineni